

Project Action Plan
Local Business Review Analyzer

Team Member	Roles & Responsibilities
Priyal Aggarwal	NLP, ML Engineer, Documentation, Data Collection
Rishabh Srivastav	Data Engineer, Data Analyst, Visualizer, Enhanced Streamlit dashboard
Kanika Jain	Data collection, Streamlit dashboard
Md Faizan	Data collection
Pranjal	<i>No significant project contribution.</i>

1. Project Overview

Project Title

Local Business Review Analyzer: Sentiment and Rating Prediction from User Feedback

Objectives & Scope

The goal of this project is to build an automated data analytics and machine learning pipeline that turns unstructured customer reviews from local businesses into actionable business intelligence.

Stage	Process	Description / Techniques Used
1. Manual Data Collection	Review Dataset Creation	Collect and compile 500–1,000+ local restaurant reviews from platforms such as Google and JustDial.
2. Data Preprocessing	Text Cleaning and Preparation	Apply tokenization, lemmatization, stop-word removal, and other text cleaning techniques to prepare reviews for analysis.
3. Predictive Modeling	Machine Learning Models	Build models for: • Sentiment Classification using Logistic Regression / BERT • Rating Prediction using Random Forest / XGBoost
4. Topic Discovery	Review Theme Extraction	Apply Latent Dirichlet Allocation (LDA) to identify common topics, customer concerns, and important review themes.
5. Deployment	Dashboard and Repository	Deploy results through an interactive Streamlit Web Dashboard and maintain the project using a GitHub Repository .

- **Core Objectives:**
 - Compile a dataset of **500 to 1,000+ local customer reviews** across business categories (cafes, salons, gyms, restaurants).
 - Preprocess and vectorize unstructured text using **TF-IDF n-grams** and **dense BERT embeddings**.
 - Train supervised models to classify sentiment (**Positive, Neutral, Negative**).
 - Train regression models to forecast numerical ratings (**1 to 5 stars**) directly from textual feedback.

- Deploy **LDA topic modeling** to automatically surface operational issues (e.g., pricing, cleanliness, wait time).
- Present insights via a **Streamlit analytics dashboard** and maintain a clean **GitHub repository**.

Team Roles & Responsibilities

Role	Key Responsibilities	Primary Tools
Data Engineer / Collector	Manually gather reviews, clean schema headers, remove noise, export structured CSVs.	Excel/CSV, Pandas, Python
NLP & ML Engineer	Build preprocessing pipelines, train sentiment classifiers, train regression models.	Scikit-Learn, NLTK, Transformers
Data Analyst & Visualizer	Perform exploratory analysis, word cloud generation, topic modeling setup.	Seaborn, Gensim, Matplotlib
Full-Stack / App Developer	Design Streamlit dashboard UI, handle model loading, maintain GitHub repository structure.	Streamlit, Git, GitHub

2. Detailed Timeline (4-Week Execution Plan)

Week	Activities	Description
Week 1	Data Collection & Exploratory Data Analysis (EDA)	Collect review data, clean and organize the dataset, and perform exploratory analysis to understand review patterns, ratings, and business-level insights.
Week 2	NLP Preprocessing, Text Vectorization & Baseline Models	Perform text preprocessing including cleaning, tokenization, and lemmatization. Apply vectorization techniques such as TF-IDF and build initial machine learning models.
Week 3	Advanced ML/AI Modeling & Topic Mining	Develop advanced models such as BERT/XGBoost, evaluate model performance, and apply topic modelling techniques like LDA to discover review themes.

Week	Activities	Description
Week 4	Streamlit Dashboard Deployment, GitHub Setup & Final Report	Create an interactive Streamlit dashboard, organize the project repository on GitHub, and complete the final documentation and report.

Daily Task Breakdown

1.Week 1: Data Gathering, Structuring, and EDA: Milestone M1: Dataset finalized and cleaned.

- **Day 1–2:** Identify target local businesses across categories (cafes, salons, gyms) on Google Maps, JustDial, and Zomato.
- **Day 3–4:** Manually extract review fields (business_name, location, review_text, rating, date) into a unified CSV format. Validate 500–1,000+ entries.
- **Day 5:** Perform exploratory data analysis (EDA): analyze star rating distributions, review lengths, and category breakdowns.
- **Day 6–7:** Handle missing values, filter extreme anomalies, and generate initial visual distributions using Seaborn and Matplotlib.

2.Week 2: Preprocessing, Feature Engineering, & Baselines: Milestone M2: Feature pipeline & baseline models trained.

- **Day 8–9:** Build text normalization functions (lowercasing, punctuation stripping, stop-word removal using NLTK).
- **Day 10:** Implement WordNetLemmatizer to reduce tokens to root dictionary forms. Assign baseline VADER sentiment scores.
- **Day 11:** Generate TF-IDF vector matrices with unigram and bigram ranges.
- **Day 12–13:** Train baseline classifiers (Multinomial Naive Bayes, Logistic Regression) for sentiment prediction.
- **Day 14:** Train baseline linear regression models for star rating predictions and compute baseline RMSE.

3.Week 3: Advanced AI Modeling & Topic Mining: Milestone M3: BERT, XGBoost, and LDA models optimized.

- **Day 15–16:** Fine-tune pre-trained bert-base-uncased transformer model for sentiment classification.
- **Day 17–18:** Train tree-based ensemble algorithms (Random Forest and XGBoost Regressors) for rating prediction.

- **Day 19–20:** Construct Latent Dirichlet Allocation (LDA) models using Gensim; evaluate topic coherence (C_v) across topic counts.
- **Day 21:** Compare model performances using standardized evaluation matrices (Accuracy, F1-Score, RMSE, MAE).

4.Week 4: Application Deployment & Project Delivery: Milestone M4: Final app deployed & documentation published.

- **Day 22–23:** Develop interactive Streamlit UI for real-time text input, predicted star score display, and word clouds.
- **Day 24:** Integrate LDA topic visualizations and business comparison views inside Streamlit.
- **Day 25:** Organize repository structure (notebooks/, models/, app/, data/) and author README.md.
- **Day 26–27:** Compile the final academic research report and verify deliverables.
- **Day 28:** Conduct final testing, push code to GitHub, and present project deliverables.

3. Resource Requirements

Software & Environment Setup

- **Programming Engine:** Python 3.9+
- **Data Manipulation:** pandas, numpy
- **NLP & Text Processing:** nltk, spacy, gensim, transformers (HuggingFace), torch
- **Machine Learning:** scikit-learn, xgboost, vaderSentiment
- **Data Visualization:** matplotlib, seaborn, wordcloud
- **App Framework & Tools:** streamlit, jupyter, git

Component	Requirement	Details
CPU	Processor Requirement	Quad-core processor such as Intel i5/i7 or AMD Ryzen 5/7 for efficient data processing and model training.
RAM	Memory Requirement	Minimum 8 GB RAM required; 16 GB RAM recommended for tasks such as BERT fine-tuning and advanced model training.
Storage	Disk Space Requirement	At least 10 GB free storage required for datasets, transformer model weights, and other project files.

Component	Requirement	Details
Dataset	Data Format & Size	CSV format dataset containing 500–1,000+ manually verified review records for analysis and model development.

4. Risk Assessment & Mitigation Strategies

Risk	Problem	Mitigation Strategy
1. Class Imbalance	The dataset may contain an unequal distribution of ratings, for example, a large number of 5-star positive reviews and fewer 1–2 star reviews, which can affect model performance.	Apply techniques such as SMOTE oversampling or use class-weighted loss functions to improve model learning across all rating classes.
2. Noise & Local Slang / Code-Switching	Reviews may contain informal language, spelling variations, or mixed languages, such as <i>"Food was banger, kaafi costly"</i> , making text understanding difficult.	Use custom stop-word filtering , text normalization techniques, and BERT sub-word tokenization to handle informal and varied language patterns.
3. Rating Discrepancies	Some reviews may contain a mismatch between the written text and the given rating, such as negative comments paired with a 5-star rating.	Verify sentiment consistency by cross-checking review polarity using tools such as VADER sentiment analysis before model training.
4. Transformer Computational Overhead	Transformer models like BERT require high computational resources and may take a long time to train on CPUs.	Use lightweight models such as DistilBERT or run training on platforms with GPU support, such as Google Colab GPU environments .

Potential Risk	Severity	Impact	Mitigation Strategy
Class Imbalance	High	Model ignores negative feedback	Apply synthetic oversampling (SMOTE) or assign class weights during model training.

Potential Risk	Severity	Impact	Mitigation Strategy
Noisy & Informal Text	Medium	Low vectorization quality	Combine regex cleaning, custom dictionary mapping, and sub-word BERT tokenization.
Mismatched Ratings	Low	High regression errors	Identify and prune outliers where VADER sentiment sharply conflicts with star ratings.
Hardware Constraints	Medium	Slower training iterations	Utilize free Cloud GPUs (Google Colab / Kaggle) or swap to lightweight DistilBERT.

5. Deliverables Checklist

- **Deliverable 1: Academic Research Report**
 - Fully formatted PDF/Markdown covering Abstract, Methodology, Models, and References.
- **Deliverable 2: Project Action Plan Document**
 - Comprehensive project roadmap, resource specifications, and risk matrices.
- **Deliverable 3: Cleaned CSV Dataset**
 - Manually verified file containing 500–1,000+ structured review entries (business_name, location, review_text, rating, date).
- **Deliverable 4: EDA & Preprocessing Notebook (01_eda_and_preprocessing.ipynb)**
 - Visualizations, cleaning code, VADER sentiment tagging, and word clouds.
- **Deliverable 5: Sentiment Classification & Rating Models (02_modeling.ipynb)**
 - Training scripts and evaluations for Logistic Regression, Naive Bayes, BERT, Random Forest, and XGBoost.
- **Deliverable 6: Topic Modeling Notebook (03_topic_modeling.ipynb)**
 - LDA theme extraction code and topic coherence visualization.
- **Deliverable 7: Interactive Streamlit Web Application (app.py)**
 - Deployed dashboard displaying real-time predictions, rating forecasts, and pain-point summaries.

- **Deliverable 8: GitHub Repository**
 - Clean directory structure with documentation (README.md, requirements.txt).

6. Success Metrics

To validate model efficacy and business value, the project measures performance against target metrics:

Evaluation Metrics Framework

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

|

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

|

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Task Domain	Metric	Target Threshold	Industry Benchmark
Sentiment Classification	Accuracy / F1-Score	> 88% Accuracy, > 0.85 F1	BERT achieves ~90–93% on domain text.
Rating Prediction	RMSE / MAE	RMSE < 0.60, MAE < 0.45	Tree ensembles achieve RMSE ~0.50–0.60.
Topic Modeling	Coherence Score (C_v)	$C_v > 0.50$	High topic interpretability.
System Delivery	Usability / App Latency	Inference time < 2 seconds	Interactive Streamlit experience.